

FastAPI 기반 LLM 백엔드 API 개발과 스트리밍 실습

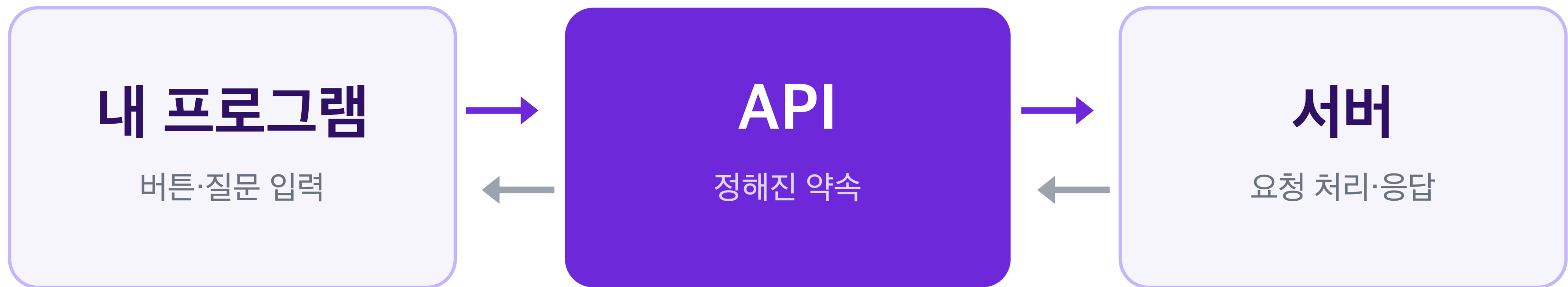
01 백엔드 API 기초

API와 HTTP 요청·응답 구조 이해



API란?

- 프로그램끼리 **정해진 방식으로 데이터를 주고받기 위한 약속**
- 사용자가 버튼을 누르거나 질문을 입력하면, 그 내용은 API 요청으로 서버에 전달됨
- 서버는 요청을 처리한 뒤, **정해진 형식으로 응답을 반환함**





API가 사용되는 예시

- 날씨 앱은 날씨 데이터를 가진 서버에 요청을 보내고, 받은 결과를 화면에 표시
- 지도 앱은 경로 계산을 서버에 요청하고, 응답 결과를 지도 위에 표시
- 챗봇은 사용자의 질문을 서버로 보내고, 생성된 답변을 다시 화면에 표시



날씨 앱

날씨 데이터를 받아옴



지도 앱

지도·위치 정보를 요청



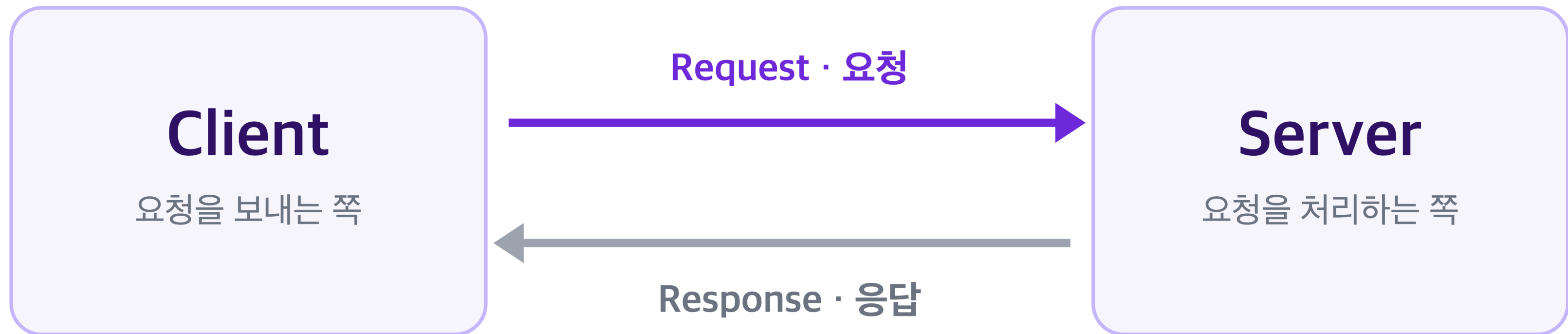
챗봇 화면

LLM 답변을 주고받음



API의 기본 동작

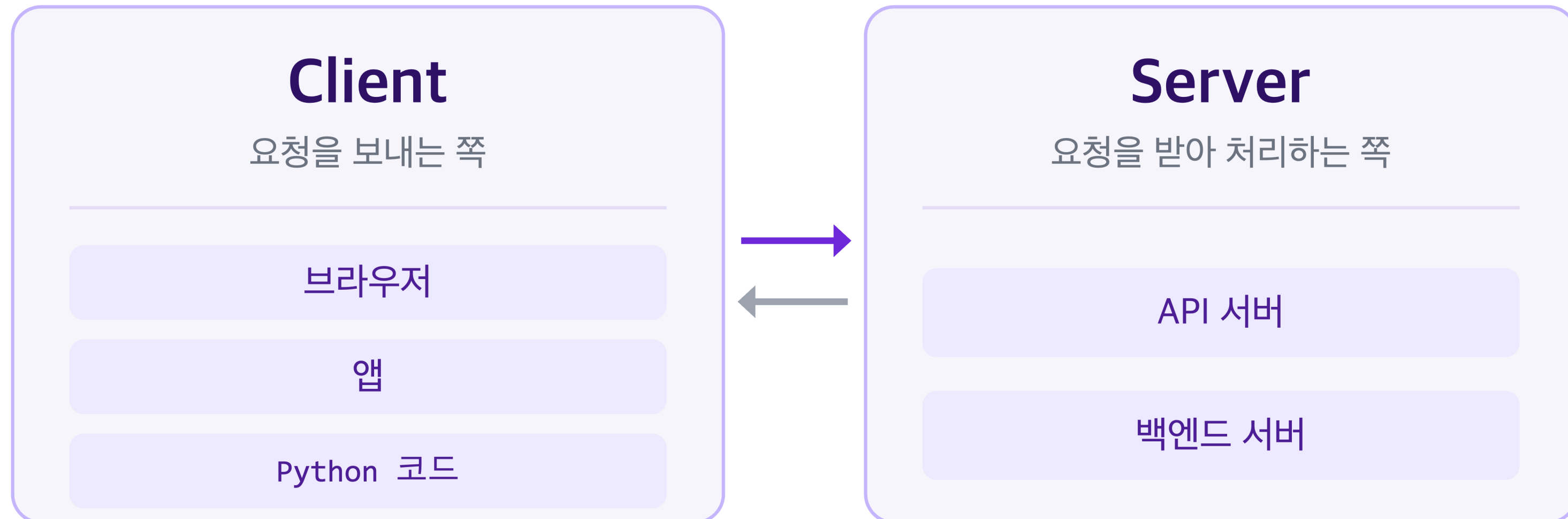
- 요청을 보내고, 처리 결과를 응답으로 받는 구조





Client와 Server

- Client와 Server는 요청 방향에 따라 정해지는 역할
- Python 코드도 요청을 보내면 Client 역할
- 이번 과정에서는 요청을 받아 처리하는 Server 구현에 집중





Backend의 역할

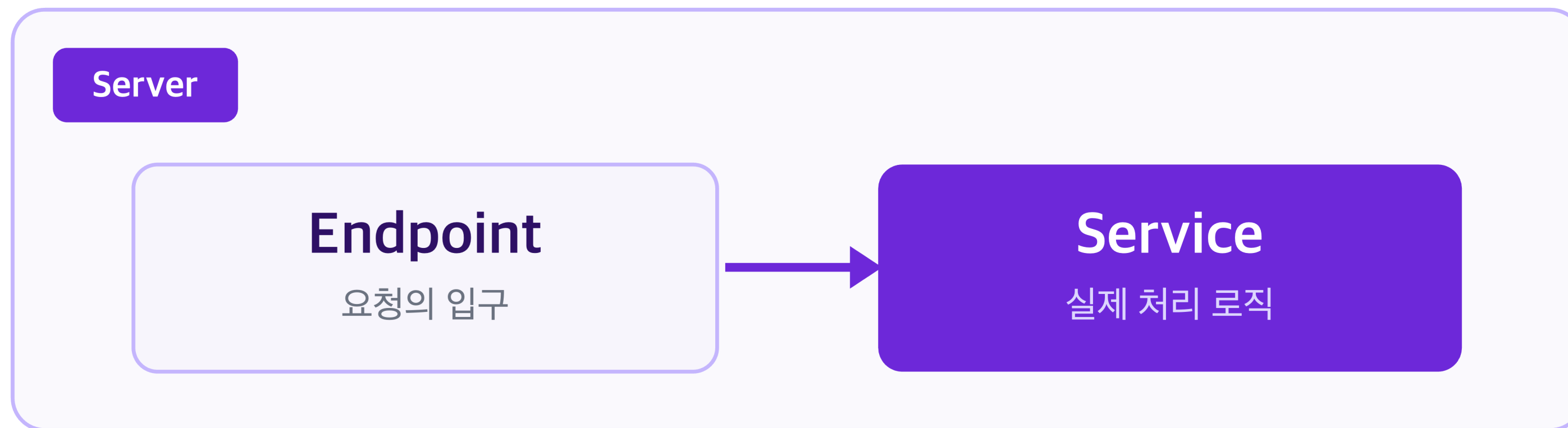
- 잘못된 입력을 그대로 처리하지 않기 위한 **중간 점검 필요**
- 외부 API 호출 결과도 **최종 응답 형식**에 맞게 정리





Service란 무엇인가

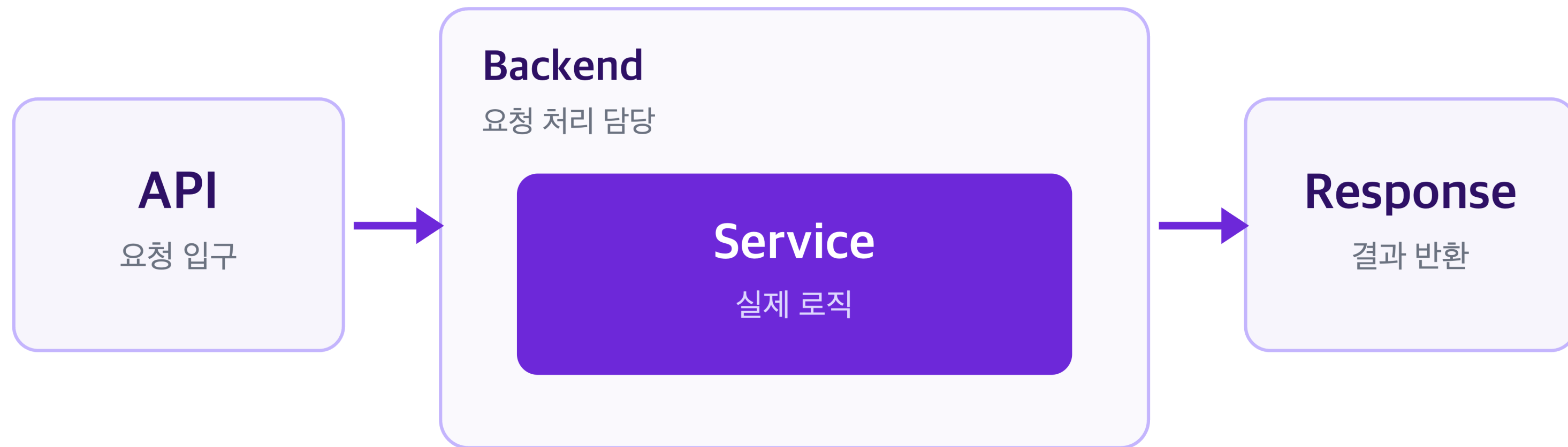
- 서버 내부에서 실제 처리를 담당하는 로직
- Endpoint가 요청의 입구라면, Service는 그 요청을 처리하는 내부 작업
- 지금은 “서버 안에서 실제 일을 하는 함수나 로직” 정도로 이해





API · Backend · Service 관계

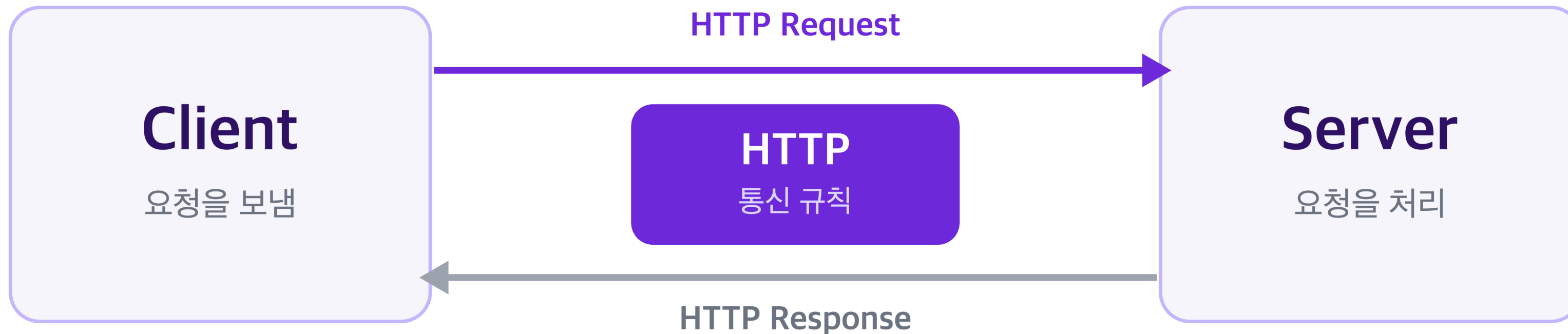
- 사용자는 내부 구조를 몰라도 API 형식만 맞추면 요청 가능
- Backend는 외부 요청을 내부 처리 흐름으로 연결
- Service가 바뀌어도 API 형식은 유지될 수 있음





HTTP란?

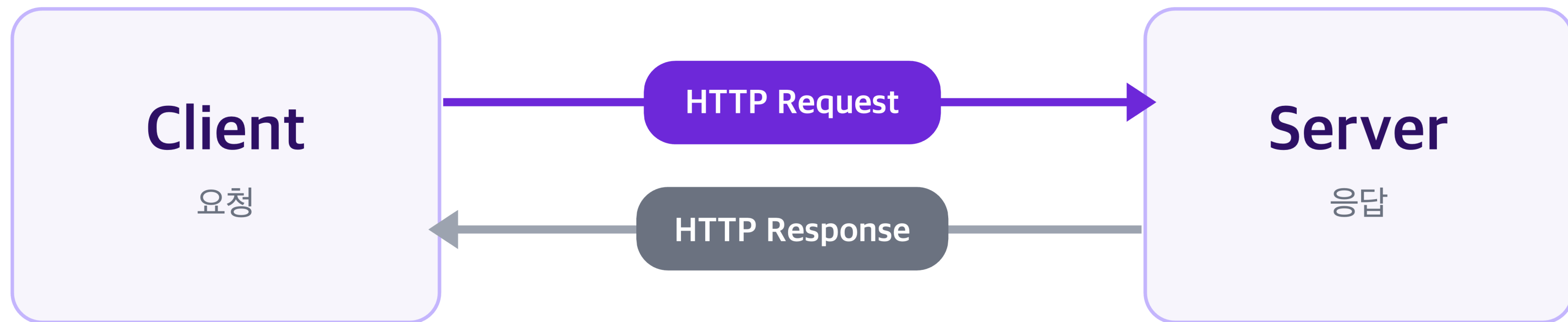
- 웹에서 클라이언트와 서버가 데이터를 주고받기 위한 **통신 규칙**
- 클라이언트는 HTTP Request를 보내고, 서버는 HTTP Response를 반환
- 대부분의 웹 API는 HTTP를 기반으로 동작





HTTP: Request와 Response

- HTTP 통신을 **요청과 응답의 왕복 구조**로 이해할 수 있음





HTTP Request의 구성 요소

- 요청을 해석할 때는 주소만 보지 않고 Method와 함께 확인
- Body가 필요한 요청인지, Header 정보가 필요한 요청인지 함께 판단
- 하나의 요청은 "무엇을 / 어디에 / 어떤 데이터로" 보낼지 담는 구조

Method

어떤 작업을 요청할지 나타냄

URL

어떤 주소로 요청할지 나타냄

Header

요청에 대한 부가 정보를 담음

Body

서버로 전달할 실제 데이터를 담음



HTTP Method

Method	설명	예시
GET	데이터 조회	GET /posts/1
POST	데이터 전송	POST /chat
PUT / PATCH	데이터 수정	PATCH /users/1
DELETE	데이터 삭제	DELETE /posts/1



GET과 POST 비교

- 데이터를 가져오는 상황과 보내는 상황을 먼저 구분
- 사용자 질문처럼 서버가 처리해야 할 입력이 있으면 POST가 자연스러움
- 요청 목적에 따라 Body 사용 여부가 달라짐

GET

데이터를 가져올 때

이럴 때 사용해요

- 게시글 조회 · 목록 보기
- 서버 상태 확인

요청 형태

Body 없이 URL만으로 요청

POST

데이터를 보낼 때

이럴 때 사용해요

- 질문 · 메시지 보내기
- 새 글 작성 · 데이터 등록

요청 형태

Body에 JSON 데이터를 담아 전송

LLM에 질문을 보내는 /chat 요청도 보통 POST 방식을 사용합니다



URL 예시

- 어떤 API 기능을 호출할지를 나타냄

/health

→ 서버 상태 확인

/chat

→ 챗봇 대화 요청

/summarize

→ 텍스트 요약 요청

/users/1

→ 특정 사용자 조회



Header 예시

- 요청이나 응답에 대한 **부가 정보**를 담음

● ● ● HTTP Header

```
Content-Type: application/json  
Authorization: Bearer <API_KEY>
```

Content-Type

보내는 데이터의 형식을 알려줌

Authorization

API 키로 인증 정보를 전달



Body 예시

- POST 요청에서는 Body에 실제 입력 데이터를 담는 경우 많음
- 사용자의 입력값이 요청 Body에 담겨 서버로 전달

● ● ● Request Body (JSON)

```
{  
  "message": "FastAPI가 뭐야?"  
}
```

- 서버에 실제로 전송하는 데이터 — JSON 형식으로 담아 보냅니다



HTTP Response의 구성 요소

- 요청 Body가 서버로 보내는 입력이라면, 응답 Body는 **서버가 돌려주는 결과**
- Status Code**를 먼저 **확인**해야 성공과 실패를 구분 가능

Status Code

응답의 성공·실패를 나타내는 숫자

200 성공 · 404 없음 · 500 서버 오류

Header

응답에 대한 부가 정보

Content-Type 등 메타데이터

Body

실제 결과 데이터

LLM 답변 · JSON 등 핵심 내용



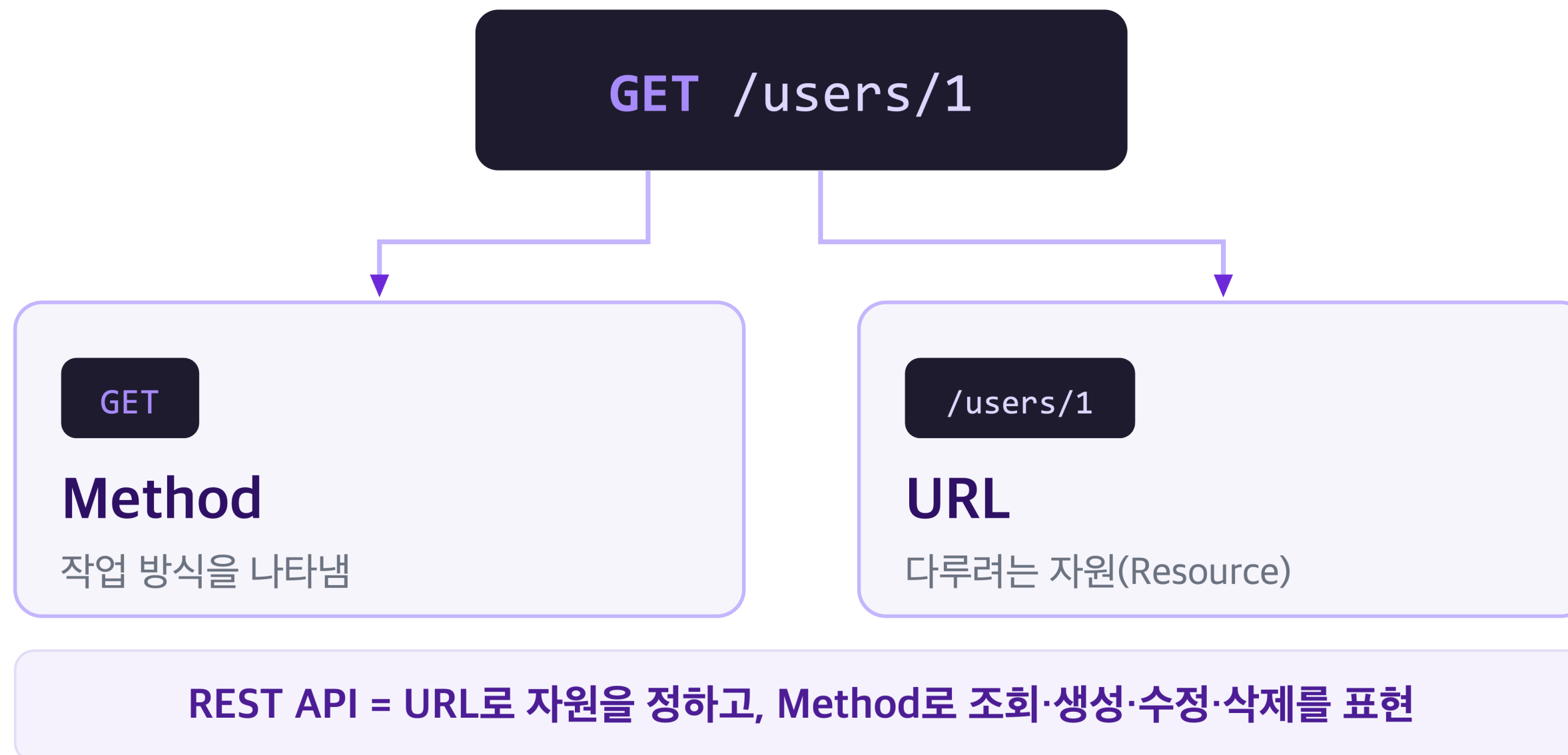
자주 보는 Status Code

코드	의미	상황
200	성공	요청이 정상 처리됨
201	생성 성공	POST 요청으로 새 데이터가 생성됨
400	잘못된 요청	요청 형식이나 값이 잘못됨
404	찾을 수 없음	요청한 URL이 없음
422	검증 실패	입력 데이터 형식이 맞지 않음
500	서버 오류	서버 내부 처리 중 오류 발생



REST API란?

- HTTP를 기반으로 클라이언트와 서버가 데이터를 주고받는 API 설계 방식
- URL은 다루려는 대상(Resource)을 나타내고, Method는 작업 방식을 나타냄
- 웹 서비스에서 조회 · 생성 · 수정 · 삭제 기능을 표현할 때 자주 사용됨





REST API의 기본 생각

- 같은 URL도 **Method**에 따라 요청 의미 변화
- API 요청은 대상과 작업을 함께 읽어야 정확히 해석 가능

Resource · 자원

URL이 가리키는 대상

/users

사용자 정보

/posts

게시글

/orders

주문 내역

URL 하나하나가 곧 '자원'

Method · 행동

자원에 무엇을 할지

GET

→ 조회 가져오기

POST

→ 생성 만들기

PATCH

→ 수정 바꾸기

DELETE

→ 삭제 지우기



REST API 예시

요청	의미
GET /posts	게시글 목록 조회
GET /posts/1	1번 게시글 조회
POST /posts	새 게시글 작성
PATCH /posts/1	1번 게시글 수정
DELETE /posts/1	1번 게시글 삭제



LLM 서비스 Endpoint 예시

- 게시글 API와 같은 요청·응답 구조는 LLM 서비스에서도 그대로 적용
- 다루는 대상이 달라도 요청·응답 구조는 동일
- 사용자의 질문도 Method, URL, Body를 갖춘 HTTP 요청으로 전달

Endpoint	역할
POST /chat	사용자 질문에 대한 답변 생성
POST /summarize	입력 문서나 텍스트 요약
GET /health	서버가 정상 동작하는지 확인

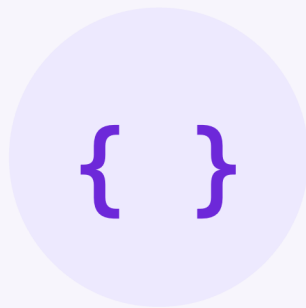
게시글 API와 같은 요청·응답 구조는 LLM 서비스에서도 그대로 적용됩니다

JSON 데이터 구조와 전달 방식 이해



JSON이란?

- API에서 데이터를 주고 받을 때 많이 사용하는 데이터 형식
- 데이터를 **Key**와 **Value**의 쌍으로 표현
- HTTP 요청 Body와 응답 Body에서 입력값과 처리 결과를 담는 데 자주 사용됨



Key-Value 쌍

이름표와 값으로 표현



가벼운 텍스트 형식

API 데이터 교환 표준



요청·응답 Body

입력값과 결과를 담음



JSON 기본 구조

- 전체 데이터는 중괄호 { } 로 표현
- Key와 Value는 콜론 : 으로 연결
- 여러 쌍은 쉼표 , 로 구분

● ● ● example.json

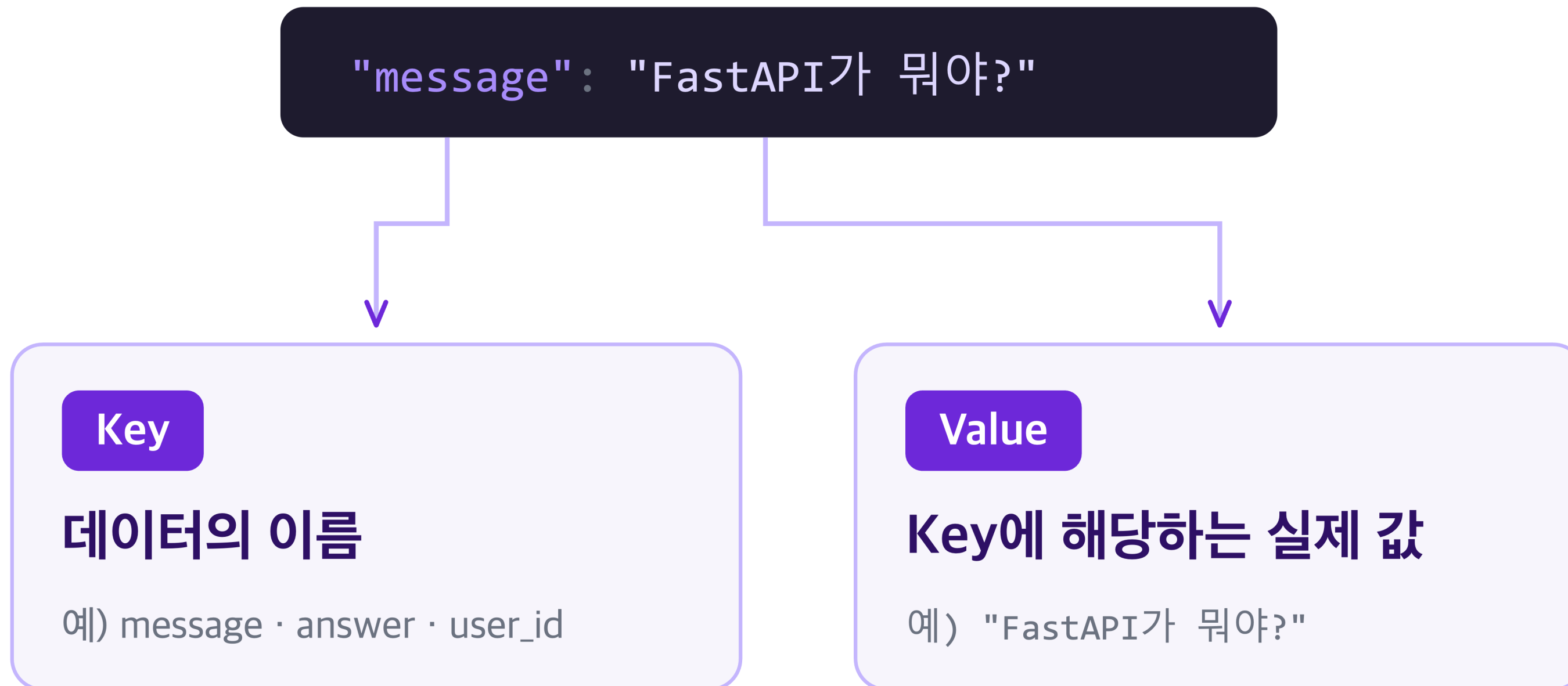
```
{  
  "name": "Elice",  
  "course": "FastAPI"  
}
```

- 중괄호 { } 로 전체를 감싸고, 각 항목은 "Key": Value 형태이며 쉼표(,)로 구분합니다



Key와 Value

- Key를 기준으로 원하는 Value를 꺼내 사용
- 서버는 약속된 Key 이름으로 입력값을 해석





요청 JSON 예시

- 요청 JSON에는 서버가 처리할 입력값이 들어감
- 사용자가 보낸 내용이 message 값으로 전달
- 서버는 message Key에서 처리할 문장을 꺼내 사용

REQUEST

클라이언트가 서버로 보내는 입력값

request.json

```
{  
  "message": "오늘 수업 내용을 요약해줘"  
}
```

- 서버가 처리할 입력값(질문·명령 등)을 담습니다



응답 JSON 예시

- 응답 JSON에는 서버의 **처리 결과**가 들어감
- 서버가 만든 결과가 **answer** 값으로 반환
- 클라이언트는 **answer Key**에서 화면에 보여 줄 문장을 꺼내 사용

RESPONSE

서버가 클라이언트로 돌려주는 처리 결과

● ● ● response.json

```
{  
  "answer": "오늘 수업은 API 요청과 응답 구조를 다뤘습니다."  
}
```

- 서버의 처리 결과(LLM 답변 등)를 담습니다



요청 JSON과 응답 JSON





직렬화

- 직렬화는 프로그램 안의 데이터를 **전송 가능한 형태로 바꾸는** 과정
- 예를 들어 Python dict를 HTTP body에 담아 보내기 위해 JSON 문자열 형태로 변환함
- 데이터를 **보낼 때** 일어나는 변환

직렬화 · Serialization

Python dict

프로그램 안의 데이터

```
{ "msg": "..." }
```

JSON 문자열

전송 가능한 텍스트

```
'{ "msg": "..."}'
```

HTTP Body

요청에 실어 전송

```
POST /chat
```

프로그램 안의 데이터를 전송 가능한 형태(JSON 문자열)로 바꾸는 과정



역직렬화

- 역직렬화는 받은 JSON 데이터를 **프로그램 안에서 다룰 수 있게 바꾸는** 과정
- 예를 들어 서버 응답 Body의 JSON 문자열을 Python dict처럼 다룰 수 있는 형태로 변환함
- 데이터를 **받을 때** 일어나는 변환

역직렬화 · Deserialization



받은 JSON 데이터를 프로그램에서 다룰 수 있는 형태(dict)로 바꾸는 과정



Python dict 예시

- Python에서는 JSON과 비슷한 데이터를 dict로 다루는 경우가 많음
- JSON은 전송 형식, dict는 Python 내부 자료형
- Key를 사용해 필요한 Value를 꺼내 사용
- API 요청 전후로 JSON과 dict 사이 변환 발생

python

```
data = {  
    "message": "FastAPI가 뭐야?"  
}  
  
print(data["message"])
```

- JSON은 데이터를 주고받는 전송 형식, dict는 Python 안에서 사용하는 자료형입니다

HTTP 요청 및 응답 확인 (실습)



실습 목표

- HTTP 요청을 직접 보내고, 응답의 Status Code, Header, Body, JSON 구조를 확인하며 해석

실습 1: HTTP 요청 및 응답 확인

실습 목표

이번 실습에서는 Python `requests` 를 사용해 HTTP 요청을 보내고 응답을 확인합니다.

단순히 코드를 보는 것이 아니라, 직접 실행하면서 다음을 관찰합니다.

- GET 요청과 POST 요청의 차이
- Status Code의 의미
- Header와 Body의 역할
- JSON 응답을 Python 객체로 다루는 방법
- Query Parameter 전달 방식
- JSON Body 전송 방식
- LLM 서비스에서 사용할 수 있는 요청 Body 형태

이 실습에서는 JSONPlaceholder 테스트 API를 사용합니다. 실제 데이터가 저장되지는 않습니다.



실습 흐름

1 GET 요청 보내기

2 Status Code 확인하기

3 Header 확인하기

4 JSON 응답을 Python 객체로 변환

5 응답 요약 함수 만들기

6 Query Parameter 사용하기

7 POST 요청 보내기

8 Header 명시해서 POST 요청

9 실패 응답 확인하기

10 LLM 요청 Body 형태 살펴보기

TODO

10단계를 마친 뒤, 배운 내용을 직접 함수로 구현해 봅니다



1. GET 요청 보내기

- 실습 포인트:
 - requests.get(url) 한 줄이 클라이언트와 서버의 첫 왕복
 - 결과는 값이 아닌 response 객체
 - Status Code · Header · Body가 함께 담김
 - URL 끝의 1을 2, 3으로 바꾸면 응답이 그에 맞게 달라짐

1. GET 요청 보내기

GET 요청은 서버에서 데이터를 조회할 때 주로 사용합니다.

아래 코드는 테스트 API에서 게시물 1개를 조회합니다.

```
import requests
import json
from pprint import pprint

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url)

print(response)
```

Python

response 객체 자체를 출력하면 응답의 요약만 보입니다.
실제 응답 내용을 확인하려면 Status Code, Header, Body를 따로 살펴봐야 합니다.

```
print("Status Code:", response.status_code)
print("Content-Type:", response.headers.get("Content-Type"))
print("Body:")
print(response.text)
```

Python



2. Status Code 확인하기

- 응답의 Status Code를 확인해 요청 처리 결과를 판단
- 실습 포인트:
 - status_code가 응답에서 **가장 먼저 봐야 할 정보**
 - 정상이라면 200, 다음 단계에서 실패 케이스에서는 400/404로 바뀜
 - 응답이 왔다는 것 = 성공이 아님 — **Status Code부터 확인**

2. Status Code 확인하기

Status Code는 요청 처리 결과를 숫자로 알려줍니다.

Status Code	의미
200	요청 성공
201	생성 성공
400	잘못된 요청
404	요청한 주소를 찾을 수 없음
500	서버 내부 오류

아래 코드에서 응답이 성공인지 확인해봅니다.

```
if response.status_code == 200:  
    print("요청이 성공했습니다.")  
else:  
    print("요청이 실패했습니다.")
```

Python



3. Header 확인하기

- 실습 포인트:
 - response.headers로 서버가 함께 보낸 부가 정보 확인
 - Content-Type이 application/json이면 다음 단계의 JSON 변환을 안심하고 진행

3. Header 확인하기

Header는 요청이나 응답에 대한 부가 정보를 담습니다.

예를 들어 Content-Type 을 보면 응답 Body가 어떤 형식인지 알 수 있습니다.

```
print("Content-Type:", response.headers.get("Content-Type"))
```

Python

전체 Header도 확인해봅니다.

실제 API를 다룰 때는 인증 정보, 데이터 형식, 캐시 정책 등도 Header에 포함될 수 있습니다.

```
for key, value in response.headers.items():  
    print(f"{key}: {value}")
```

Python



4. JSON 응답을 Python 객체로 변환하기

- 응답 Body의 JSON 데이터를 Python에서 다룰 수 있는 객체로 변환
- 실습 포인트:
 - `response.json()` 호출 한 줄
 - 변환 후 `type()`으로 dict인지 직접 확인해 보면 변환 사실이 한눈에 보임
 - 응답이 항상 JSON이라 단정하지 말 것

4. JSON 응답을 Python 객체로 변환하기

응답 Body가 JSON 형식이면 `response.json()` 을 사용해 Python dict로 바꿀 수 있습니다.

```
data = response.json()

print(type(data))
pprint(data)
```

Python

dict가 되었으므로 key를 사용해 필요한 값만 꺼낼 수 있습니다.

```
print("게시글 ID:", data["id"])
print("제목:", data["title"])
print("본문:", data["body"])
```

Python



5. 응답 요약 함수 만들기

- 실습 포인트:
 - 매 응답마다 **같은 형식으로 정리** — 정상·실패를 셀 단위로 비교 가능
 - 핵심 한 줄:
 - **try 블록 안의 response.json()**
 - 외부 응답이 항상 JSON일 거라 **단정하지 않는 안전망**(try/except)
 - JSON 아닌 응답을 일부러 던져 except 가지가 동작하는지 직접 확인 권유

5. 응답 요약 함수 만들기

여러 API 요청을 반복해서 테스트할 것이므로, 응답을 보기 좋게 출력하는 함수를 만들어둡니다.

```
def print_response_summary(response):  
    print("=" * 60)  
    print("Status Code:", response.status_code)  
    print("Content-Type:", response.headers.get("Content-Type"))  
    print("-" * 60)  
  
    try:  
        body = response.json()  
        print("JSON Body:")  
        print(json.dumps(body, ensure_ascii=False, indent=2))  
    except Exception:  
        print("Text Body:")  
        print(response.text)  
  
    print("=" * 60)  
  
print_response_summary(response)
```



6. Query Parameter 사용하기

- 실습 포인트:
 - `params`에 dict 넘기면 requests가 'key=value' 형태로 자동 변환해 URL 뒤에 붙임
 - `userId=1`로 호출 — 응답의 모든 게시글이 `userId=1`인지 직접 확인
 - `params` 값을 다른 숫자로 바꾸면 **응답이 따라 달라짐**

6. Query Parameter 사용하기

Query Parameter는 URL 뒤에 붙는 입력값입니다.

예를 들어 특정 사용자의 게시글만 조회하고 싶을 때 다음과 같은 형태를 사용할 수 있습니다.

`/posts?userId=1`

Python `requests`에서는 `params` 인자로 Query Parameter를 전달합니다.

```
url = "https://jsonplaceholder.typicode.com/posts"

params = {
    "userId": 1
}

response = requests.get(url, params=params)

print_response_summary(response)
```

Python



7. POST 요청 보내기

- 실습 포인트:
 - requests.get → requests.post / json=dict로 보낼 데이터 함께 넘김
 - json= 인자가 dict를 자동 직렬화해 Body에 담음
 - 응답 Status Code가 200이 아닌 201 — POST가 만든 결과의 의미
 - 보내는 title을 바꿔 호출하면 응답에 그대로 비침 — 첫 왕복의 가장 직관적 확인

7. POST 요청 보내기

POST 요청은 서버에 데이터를 보낼 때 주로 사용합니다.

사용자 입력, 게시물 작성 내용, 챗봇 질문 등은 보통 요청 Body에 담아 서버로 전달합니다.

```
url = "https://jsonplaceholder.typicode.com/posts"

payload = {
    "title": "hello",
    "body": "API practice",
    "userId": 1
}

response = requests.post(url, json=payload)

print_response_summary(response)
```

Python

json=payload 로 전달하면 Python dict가 JSON Body로 전송됩니다.
우리가 서버로 보낸 데이터를 다시 확인해봅니다.

```
print("요청 Body로 보낸 데이터:")
print(json.dumps(payload, ensure_ascii=False, indent=2))
```

Python



8. Header를 명시해서 POST 요청 보내기

- 실습 포인트:
 - `Content-Type: application/json`을 명시 — JSON 형식임을 서버에 분명히 알림
 - `requests`의 `json=` 인자가 사실상 이 헤더를 자동으로 붙여줌
 - 1일차 이론의 `Content-Type`이 실제 호출에 어떻게 등장하는지 짚는 단계

8. Header를 명시해서 POST 요청 보내기

`json=payload`를 사용하면 `Content-Type: application/json` Header가 자동으로 설정됩니다.

이번에는 Header를 직접 지정해봅니다.

```
headers = {  
    "Content-Type": "application/json"  
}  
  
response = requests.post(  
    url,  
    headers=headers,  
    json=payload  
)  
  
print_response_summary(response)
```

Python



9. 실패 응답 확인하기

- 실습 포인트:
 - 존재하지 않는 자원(/posts/9999)으로 GET — Status Code 404 반환
 - 호출 자체는 성공했지만 결과가 '실패' — 항상 Status Code부터 확인
 - 앞 단계의 응답 요약 함수로 정상·실패 응답을 같은 형식으로 나란히 비교

```
if response.status_code == 200:  
    print("요청이 성공했습니다.")  
else:
```

9. 실패 응답 확인하기

존재하지 않는 주소로 요청을 보내면 실패 응답이 돌아옵니다.

실패 응답도 Status Code와 Body를 확인해야 원인을 추정할 수 있습니다.

```
wrong_url = "https://jsonplaceholder.typicode.com/not-found-url"  
  
response = requests.get(wrong_url)  
  
print_response_summary(response)
```

Python



10. LLM 서비스 요청 Body 형태 살펴보기

- 실습 포인트:
 - chat_request dict의 message 키에 사용자 질문 — LLM 호출의 입력 구조
 - 게시글 Body와 모양이 거의 동일 — JSON으로 데이터를 묶는 구조는 같음
 - 지금까지 클라이언트 입장이었다면, 다음 섹션부터 시선을 서버 안으로

10. LLM 서비스 요청 Body 형태 살펴보기

아직 실제 LLM API를 호출하지는 않습니다.

다만 LLM 챗 API에 사용자 질문을 보낸다면 요청 Body는 아래처럼 구성될 수 있습니다.

```
chat_payload = {  
    "message": "FastAPI가 뭐야?",  
    "temperature": 0.7  
}  
  
print(json.dumps(chat_payload, ensure_ascii=False, indent=2))
```

Python

요청 Body는 결국 서버에 전달할 입력 데이터입니다.

LLM 서비스에서는 `message`에 사용자의 질문이 들어가고, 옵션 값이 함께 전달될 수 있습니다.



TODO

- TODO 1. 게시글 하나를 조회하는 함수 만들기
- TODO 2. 조회 결과에서 필요한 정보만 추출하기
- TODO 3. POST 요청 함수 만들기
- TODO 4. LLM 챗 요청 Body 생성 함수 만들기

```
if response.status_code == 200:  
    print("요청이 성공했습니다.")  
else:  
    print("요청이 실패했습니다.")
```



실습 정리

- GET 요청은 데이터를 조회할 때 사용
- POST 요청은 서버에 데이터를 보낼 때 사용
- Query Parameter는 조회 조건을 전달할 때 사용
- JSON Body는 서버에 전달할 실제 입력 데이터를 담음
- LLM 서비스에서도 사용자의 질문은 **JSON Body 형태**로 서버에 전달

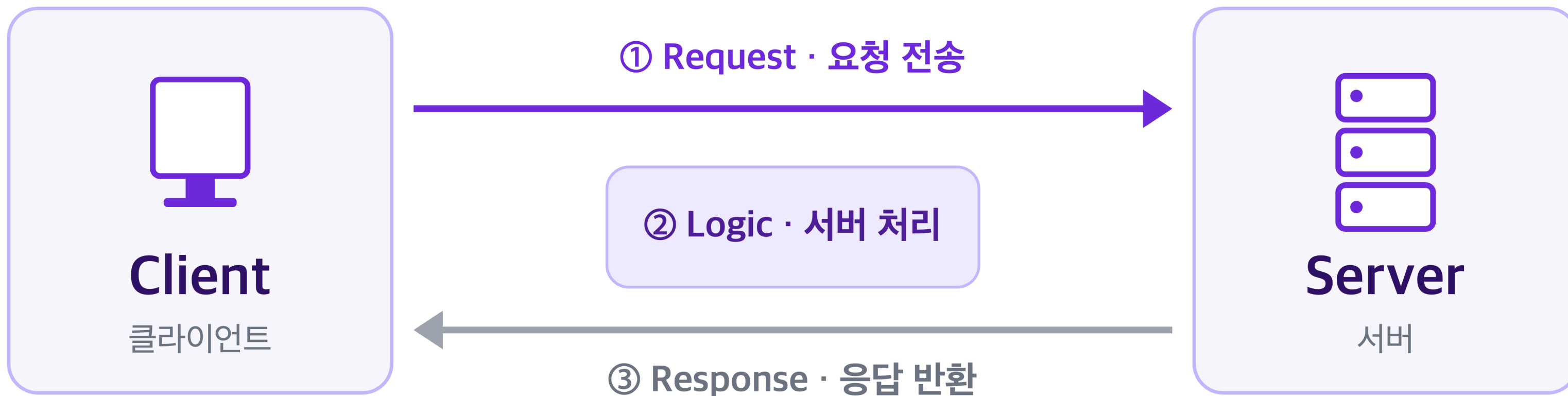
```
if response.status_code == 200:  
    print("요청이 성공했습니다.")  
else:  
    print("요청이 실패했습니다.")
```

클라이언트-서버 요청 흐름 이해



요청과 처리 흐름

- 클라이언트가 보낸 요청은 서버 안에서 처리 흐름으로 이어짐
- 서버는 **요청 정보를 확인**하고, 필요한 처리를 수행한 뒤 **응답 생성**



클라이언트의 요청은 서버 안에서 처리(Logic)를 거쳐 응답으로 돌아옵니다



요청 정보 이해

- 서버는 요청을 받으면 어떤 요청인지 먼저 확인함

Method	어떤 작업을 요청하는가
Path	어떤 기능을 호출하는가
Query	조회 조건은 무엇인가
Body	실제 입력 데이터는 무엇인가
Header	데이터 형식·인증 정보는 무엇인가

● ● ● 예시 요청

```
POST /chat
Method: POST Path: /chat
Body: {"message": "FastAPI가 뭐야?"}
Header: Content-Type: application/json
```



요청과 기능 매핑

- 서버는 **Method**와 **Path**를 보고 어떤 처리를 실행할지 결정

GET /posts/1



게시글 조회 로직

POST /chat



질문 처리 로직

GET /health



서버 상태 확인 로직

FastAPI에서는 이 매핑을 `@app.get()` · `@app.post()` 같은 코드로 작성합니다



입력 데이터 처리

- POST 요청에서 Body는 서버 내부 처리 로직의 입력값으로 사용됨
- 서버 내부에서는 필요한 값을 꺼내 처리함
- 요청 Body는 서버가 실제로 처리해야 하는 입력 데이터

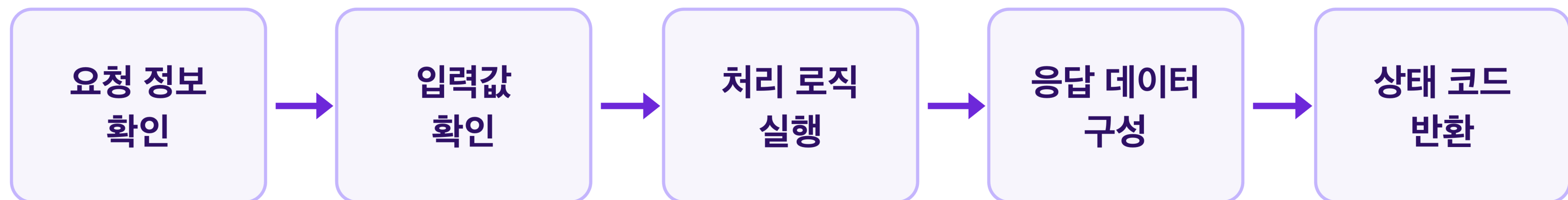




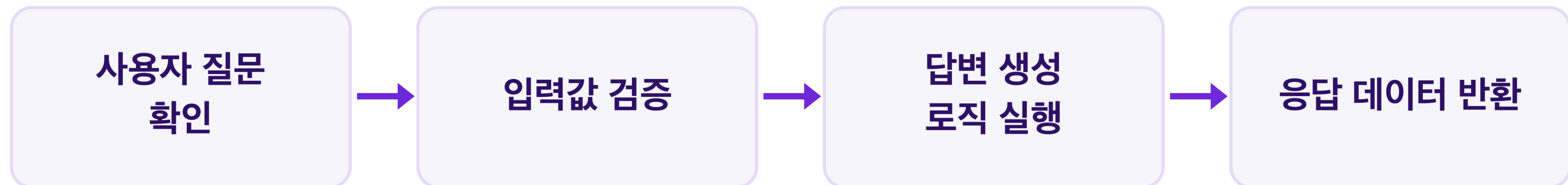
서버 처리 구조

- 서버 내부 처리는 다음 순서로 이루어짐

서버 처리 순서



예) /chat 요청 처리





응답 데이터 구성

- 서버는 처리 결과를 클라이언트가 이해할 수 있는 형태로 반환

SUCCESS

성공 응답 예시

```
{  
  "status": "ok",  
  "answer": "오늘 일정은 ..."  
}
```

ERROR

실패 응답 예시

```
{  
  "status": "error",  
  "message": "입력값이 잘못됐어요"  
}
```

요청 내용이 달라지면 서버가 만드는 응답도 달라집니다



응답 대기 구조

- 모든 요청이 바로 끝나는 것은 아님
- 일반적인 응답 방식에서는 처리 결과가 모두 만들어진 뒤 한 번에 반환됨



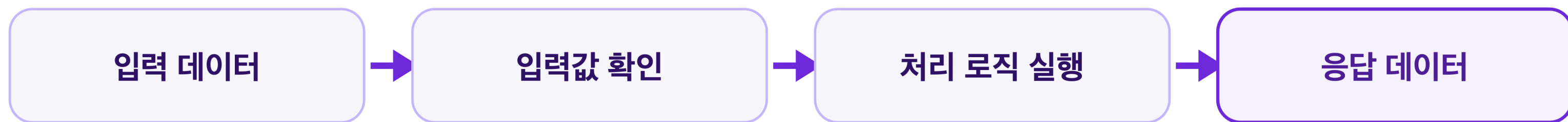
응답이 늦어질 수 있는 경우





함수 관점의 서버 로직

- 서버 내부 흐름은 하나의 함수처럼 생각할 수 있음
- Python 함수로 표현하면 다음과 같은 구조가 됨
- 요청 데이터가 들어오고, 처리 결과가 **응답 데이터로 반환**되는 구조



● ● ● handler.py

```
def handler(body):  
    if not body: return error_response()  
    result = process(body)  
    return {"status": "ok", "answer": result}
```

요청이 들어오면 검증·처리를 거쳐 응답 데이터로 반환되는 구조

기본 서버 처리 로직 구현 (실습)



handler란?

- 요청 데이터를 받아 처리하고 **응답을 만드는 함수**



handler는 요청 데이터를 받아 처리하고 응답 데이터를 만드는 함수입니다



handler가 하는 일

- handler 안에서 주로 다음 일이 일어남



이 4단계가 거의 모든 handler에 공통으로 들어가는 격



실습 목표

- 요청 데이터가 서버 내부에서 검증 · 처리 · 응답으로 이어지는 흐름을 Python 함수로 이해

실습 2. 기본 서버 처리 로직 구현

실습 목표

이번 실습은 실제 FastAPI 서버를 만들기 전에, 서버 내부에서 일어나는 처리 흐름을 Python 함수로 직접 흉내 내어 익히는 단계입니다. 진짜 HTTP 서버를 띄우지 않고도 「입력을 받아 응답을 반환하는 함수」 하나가 어떻게 자라 가는지 손에 익힙니다.

단계별로 다음을 다룹니다.

- 서버 처리 흐름 네 단계(입력 받기 · 입력값 확인 · 처리 · 응답 만들기)를 Python 함수로 구현
- 요청 데이터를 dict로 표현해 handler에 전달하기
- 입력 검증 추가와 통일된 응답 구조(`success` / `data` / `error`) 만들기
- Status Code와 Body를 함께 반환하는 형태로 다듬기
- Method + Path로 요청을 알맞은 handler에 연결하는 작은 라우터 구성



실습 흐름

1 요청 데이터를 Python dict로 표현

2 가장 단순한 handler 함수

3 입력 값이 없을 때 생기는 문제

4 입력 검증 추가하기

5 응답 형식 통일하기

6 Status Code까지 함께 표현

7 가짜 LLM 응답 함수 만들기

8 챗 API 형태의 handler 만들기

9 요청 전체 구조 흉내 내기

10 간단한 API 서버 흐름 만들기

TODO 10단계를 마친 뒤, 배운 내용을 직접 함수로 구현해 봅니다



1. 요청 데이터를 Python dict로 표현하기

- 실습 포인트:
 - JSON으로 도착할 요청 Body가 서버 안에서 **dict로 변환된 모습**
 - data['message']로 사용자 문장 꺼내기 — 다음 단계 handler에서 같은 형태 재등장
 - 1일차 1실습의 POST dict와 거의 같은 모양 — 직렬화의 반대편

1. 요청 데이터를 Python dict로 표현하기

API 요청 Body로 들어오는 JSON은 Python 코드에서는 dict처럼 다룰 수 있습니다.

아래 `request_data` 는 사용자가 서버에 보낸 요청 Body라고 생각하면 됩니다.

```
from pprint import pprint

request_data = {
    "message": "hello api"
}

print(request_data)
print(request_data["message"])
```

Python



2. 가장 단순한 handler 함수

- 실습 포인트:
 - data 한 인자만 받는 가장 작은 골격 — 입력 받아 응답 반환
 - 핵심 한 줄: `data['message']` — handler의 '값 꺼내기' 역할
 - 검증·LLM 호출은 없음 — 다음 단계부터 한 가지씩 추가

2. 가장 단순한 handler 함수

handler는 요청 데이터를 받아 처리하고 응답을 만드는 함수입니다.

아래 함수는 요청 데이터에서 `message`를 꺼내 대문자로 변환한 뒤 dict 형태로 반환합니다.

```
def handle_request(data):  
    message = data["message"]  
    result = message.upper()  
  
    return {  
        "result": result  
    }  
  
response = handle_request(request_data)  
print(response)
```

Python

같은 handler에 여러 입력을 넣어보면서, 입력에 따라 응답이 어떻게 달라지는지 확인합니다.

```
examples = [  
    {"message": "backend api"},  
    {"message": "python handler"},  
    {"message": "json response"}  
]  
  
for example in examples:  
    print("입력:", example)  
    print("응답:", handle_request(example))  
    print("-" * 40)
```

Python



3. 입력 값이 없을 때 생기는 문제

- 실습 포인트:
 - message가 빠진 dict를 handler에 넘기면 **KeyError 발생**, 함수 중단
 - 서버가 들어온 요청을 **그대로 믿으면 위험** — 검증의 필요성
 - 다음 단계에서 입력 검증으로 해결

3. 입력값이 없을 때 생기는 문제

요청 데이터에 `message`가 없으면 handler가 정상적으로 처리할 수 없습니다.

실제 서버 로직에서는 들어온 입력값을 먼저 확인해야 합니다.

```
bad_request_data = {}
```

```
# 아래 코드는 KeyError를 발생시킬 수 있으므로 주석 처리합니다.
```

```
# handle_request(bad_request_data)
```

Python



4. 입력 검증 추가하기

- 실습 포인트:
 - `data.get('message')`로 Key 없어도 None 반환 — 오류 없이 진행
 - `if not message:` 비어 있으면 곧장 오류 응답 반환 — 처리 로직 보호
 - 검증 실패도 함수가 '반환'으로 끝남 — **실패도 응답으로 돌려준다**는 첫 사례

4. 입력 검증 추가하기

`data.get("message")` 를 사용하면 key가 없을 때 오류 대신 `None` 을 받을 수 있습니다.

이제 `message` 가 없거나 비어 있으면 에러 응답을 반환하도록 바꿔봅시다.

```
def handle_request_with_validation(data):
    message = data.get("message")

    if not message:
        return {
            "error": "message 값이 필요합니다."
        }

    result = message.upper()

    return {
        "result": result
    }

print(handle_request_with_validation({"message": "hello api"}))
print(handle_request_with_validation({}))
print(handle_request_with_validation({"message": ""}))
```



5. 응답 형식 통일하기

- 실습 포인트:
 - 모든 응답에 **success·data·error** 세 키 — 정상·실패가 같은 모양
 - 받는 쪽은 success만 보고 분기 — 처리 로직 단순화
 - 이 발상이 3일차 LLM /chat 응답 구조 설계에서 다시 등장

5. 응답 형식 통일하기

API 응답 구조가 매번 다르면 클라이언트가 처리하기 어렵습니다.

성공 응답과 실패 응답을 비슷한 구조로 맞춰봅니다.

```
def handle_request_v2(data):  
    message = data.get("message")  
  
    if not message:  
        return {  
            "success": False,  
            "error": "message 값이 필요합니다.",  
            "data": None  
        }  
  
    result = message.upper()  
  
    return {  
        "success": True,  
        "error": None,  
        "data": {  
            "result": result  
        }  
    }  
  
print(handle_request_v2({"message": "hello api"}))  
print(handle_request_v2({}))
```



6. Status Code까지 함께 표현하기

- 실습 포인트:
 - 반환값을 (`status_code`, `body`) 튜플로 묶어 응답의 두 정보를 함께 표현
 - 정상 200 / 검증 실패 400 / 알 수 없는 요청 404

6. Status Code까지 함께 표현하기

실제 HTTP 응답에는 Status Code가 포함됩니다.

아직 실제 서버를 실행하지는 않지만, dict 안에 `status_code`를 넣어 HTTP 응답 구조를 흉내낼 수 있습니다.

```
def handle_request_v3(data):
    message = data.get("message")

    if not message:
        return {
            "status_code": 400,
            "body": {
                "success": False,
                "error": "message 값이 필요합니다.",
                "data": None
            }
        }

    result = message.upper()

    return {
        "status_code": 200,
        "body": {
            "success": True,
            "error": None,
            "data": {
                "result": result
            }
        }
    }

print(handle_request_v3({"message": "hello api"}))
print(handle_request_v3({}))
```



7. 가짜 LLM 응답 함수 만들기

- 실습 포인트:
 - fake_llm_response 함수로 답변 만드는 부분을 따로 분리
 - 백엔드 4단계 중 '처리 실행'이 별도 함수로 떨어져 나옴
 - 3일차에서 이 자리에 진짜 OpenAI 호출이 들어감 ('가짜'라는 이름의 이유)

7. 가짜 LLM 응답 함수 만들기

아직 실제 LLM API를 호출하지 않습니다.

대신 사용자 질문을 받아 가짜 답변을 반환하는 함수를 만들어봅니다.

```
def fake_llm_response(message):  
    return f'''{message}에 대한 가짜 LLM 응답입니다.''  
  
print(fake_llm_response("FastAPI가 뭐야?"))
```

Python



8. 챗 API 형태의 handler 만들기

- 실습 포인트:
 - chat_handler 한 함수에 요청 받기·검증·처리·반환 모두 등장
 - 지금 보는 줄이 백엔드 4단계 중 어느 단계인지 의식하며 따라가기
 - 이 형태가 2일차 FastAPI Endpoint 함수로 그대로 발전

8. 챗 API 형태의 handler 만들기

사용자 질문을 입력받고, 가짜 LLM 응답을 만들어 반환하는 handler를 만들어봅니다.

이 함수는 나중에 만들 /chat API의 내부 처리 흐름을 단순히 흉내낸 것입니다.

```
def chat_handler(data):
    message = data.get("message")

    if not message:
        return {
            "status_code": 400,
            "body": {
                "success": False,
                "error": "message 값이 필요합니다.",
                "data": None
            }
        }

    answer = fake_llm_response(message)

    return {
        "status_code": 200,
        "body": {
            "success": True,
            "error": None,
            "data": {
                "answer": answer
            }
        }
    }

print(chat_handler({"message": "FastAPI가 뭐야?"}))
print(chat_handler({}))
```




9. 요청 전체 구조 흉내 내기

- 실습 포인트:
 - request dict에 **method·path·body 세 키** — HTTP 요청의 모양을 코드로
 - Body 외에 Method·Path가 함께 있어야 다음 단계의 라우팅이 가능

9. 요청 전체 구조 흉내 내기

실제 HTTP 요청에는 Method, Path, Body가 포함됩니다.

아래처럼 요청 전체를 dict로 표현해볼 수 있습니다.

```
api_request = {  
    "method": "POST",  
    "path": "/chat",  
    "body": {  
        "message": "API 서버가 하는 일을 알려줘"  
    }  
}  
  
print(api_request)
```

Python



10. 간단한 API 서버 흐름 만들기

- 실습 포인트:
 - if method+path == 'POST /chat'이면 chat_handler — 1일차 4선의 '요청.기능 매핑'
 - 그 외 요청은 404 — 1일차 이론의 404가 직접 만들어 돌려주는 응답이 됨
 - 이 if 문이 2일차 FastAPI 데코레이터 한 줄로 사라짐

10. 간단한 API 서버 흐름 만들기

method와 path를 확인한 뒤 알맞은 handler로 요청을 넘기는 구조를 만들어봅니다.

```
def simple_api_server(request):
    method = request.get("method")
    path = request.get("path")
    body = request.get("body", {})

    if method == "POST" and path == "/chat":
        return chat_handler(body)

    return {
        "status_code": 404,
        "body": {
            "success": False,
            "error": "요청한 API를 찾을 수 없습니다.",
            "data": None
        }
    }

print(simple_api_server(api_request))
print(simple_api_server({"method": "GET", "path": "/unknown", "body": {}}))
```



TODO

- TODO 1. 입력 검증 함수 만들기
- TODO 2. 공통 응답 생성 함수 만들기
- TODO 3. 챗 handler 다시 만들기
- TODO 4. /chat 요청을 처리하는 서버 함수 만들기



실습 정리

- 요청 데이터는 dict로 표현 가능
- handler는 입력을 받고 처리 후 응답을 반환
- 입력 검증은 API 안정성의 기본
- 응답 구조를 통일하면 클라이언트가 결과를 해석하기 쉬워짐
- 오늘 만든 handler 구조는 이후 실제 **FastAPI Endpoint 구현으로 확장** 가능